# TOWARDS A UNIFIED VIEW OF SPARSE FEED-FORWARD NETWORK IN TRANSFORMER

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

Large and sparse feed-forward networks (S-FFN) such as Mixture-of-Experts (MoE) have demonstrated to be an efficient approach for scaling up Transformers model size for *pretraining*. By only activating part of the FFN parameters conditioning on input, S-FFN improves generalization performance while keeping training and inference cost (in FLOPs) fixed. A growing body of work has been focusing on improving the S-FFN design including routing method and load balancing in the context of MoEs. In this work, we make connections between S-FFN and sparse neural memory and present a unified framework to categorize design choices along two axes: memory block size and memory block selection method. Using this unified framework, we compare several S-FFN architectures on language modeling and provide insights into their relative efficacy and efficiency. For example, smaller memory block size leads to lower perplexity with potentially little extra FLOPs; selection through a gate in general improves the perplexity-FLOPs trade-off but has worse perplexity than selection uses hidden states without gate. Based on these insights, we propose a new selection method — **`Avg-K`** that selects blocks through mean aggregated hidden state statistics and with gate. With 1% additional FLOPs, **`Avg-K`** achieves 2.16 lower perplexity than a vanilla transformer, while Switch Transformer 0.51 lower.

## 1 INTRODUCTION

Large-scale pretrained language models (LMs) achieve remarkable performance and generalization ability for NLP tasks (Radford & Narasimhan, 2018; Devlin et al., 2019; Liu et al., 2019; Radford et al., 2019; Brown et al., 2020; Raffel et al., 2020; Chowdhery et al., 2022). Scaling up model size (the number of parameters) has been shown as a reliable recipe to achieve better generalization as well as unlock new capabilities (Kaplan et al., 2020; Chowdhery et al., 2022; Hoffmann et al., 2022; Wei et al., 2022). The computational resources required to train larger language models become formidable for many in the research community (Schwartz et al., 2020; Zhang et al., 2022).

Such dilemma has motivated research into more efficient model designs (Lepikhin et al., 2021; Du et al., 2021) that allow increased number of parameter while keeping down the required compute. Specifically, most approaches focus on scaling up transformer's feed-forward network with sparsely activated parameters, thus a scaled and sparse FFN (S-FFN). There have been two major approaches for achieving sparse activation (Fig. 1; §2.2): treating S-FFN as a neural memory (Sukhbaatar et al., 2015a), a sparse memory efficiently retrieves and activates only parts of the memory cells (Lample et al., 2019); or a Mixture-of-Expert Network (MoE) (Lepikhin et al., 2021; Du et al., 2021; Roller et al., 2021; Lewis et al., 2021; Chi et al., 2022) replaces a FFN with several equal-sized ones and activates only a few.

Although both memory and MoE models achieve S-FFN, they have been considered as two completely different approaches. We aim to draw the connections between these two classes of S-FFN: What key design choices do they have in common? Which design choices are essential for their generalization and computation efficiency? Can the effective ingredients of each method be transferred and combined to further improve performance?

In order to answer these questions, we start from the neural memory view of FFN (Sukhbaatar et al., 2015a) (§2.1) and reduce all S-FFN to the same mathematical form (§3.1). Then, we characterize

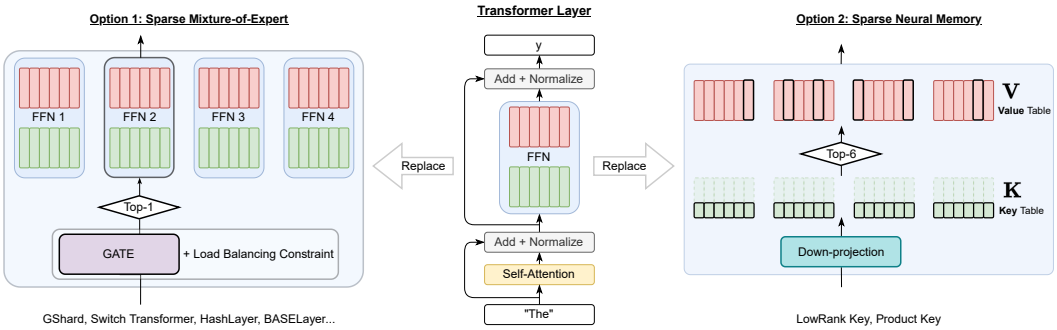



Figure 1: Sparse Mixture-of-Expert and Sparse Neural Memory as two different methods.

various S-FFN methods along two dimensions — memory block size (e.g., expert size) and memory block selection method (e.g., gate) (§3.2). Using this framework, we made following contribution:

- We study a wide range of memory block sizes besides common block size in MoEs (Fedus et al., 2022) and show that, compared with a larger one, a smaller block size can keep improving the perplexity over baseline with potentially little incurred FLOPs (§5.1 §5.2), leading to better perplexity/computation trade-offs.
- We conduct a systematic exploration of memory block selection methods to discover their relative efficacy (§5.2). Specifically, we find selection through a gating function in general improves the Pareto frontier but the current gating parameterization in MoEs has worse perplexity than selection that doesn't have gate and uses hidden states.
- Drawing on insights above, we propose a simple gate for S-FFN— **`Avg-K`** (§6), which efficiently selects based on the mean aggregated hidden states of each block. With 1% additional FLOPs, **`Avg-K`** achieves 2.16 lower perplexity than a vanilla transformer, while Switch Transformer 0.51 lower. As an additional bonus, it is effective without load balancing constraints, while conventional one degrades (Shazeer et al., 2017; Eigen et al., 2014).

## 2 Background

### 2.1 Feed-Forward Network

A transformer layer (Vaswani et al., 2017) is made of a self-attention block and a Feed-Forward Network (FFN) block. FFN receives an input vector $\mathbf{x} \in \mathbb{R}^d$ from the self-attention block, multiplies it with $\mathbf{K} \in \mathbb{R}^{d_m \times d}$, applies an non-linear function to obtain the hidden states $\mathbf{m} \in \mathbb{R}^{d_m}$ and applies another affine transformation $\mathbf{V} \in \mathbb{R}^{d_m \times d}$ to produce a $d$-dimensional output (Eq. 1).

$$\text{FFN}(\mathbf{x}) = f(\mathbf{x} \cdot \mathbf{K}^\top) \cdot \mathbf{V} = \mathbf{m} \cdot \mathbf{V} \in \mathbb{R}^d \quad \text{(Multi-Layer Perceptron view)} \tag{1}$$

$$\text{FFN}(\mathbf{x}) = \sum_{i=0}^{d_m-1} f(\mathbf{x} \cdot \mathbf{k}_i) \cdot \mathbf{v}_i = \sum_{i=0}^{d_m-1} m_i \cdot \mathbf{v}_i \in \mathbb{R}^d \quad \text{(Neural Memory view)} \tag{2}$$

FFN is a multi-layer perceptron, but it can also be described as a neural memory (Sukhbaatar et al., 2015b; 2019; Geva et al., 2021)(Eq. 2). In this view, FFN consists of $d_m$ key-value pairs, called *memory cells*. Each key is represented by a $d$-dimensional $\mathbf{k}_i \in \mathbb{R}^d$, and together form the *key table*; likewise, a *value table* $\mathbf{V} \in \mathbb{R}^{d_m \times d}$. When the memory receives the query input $\mathbf{x} \in \mathbb{R}^d$, it multiplies $\mathbf{x}$ with every $\mathbf{k}_i$; followed by the non-linear function, it produces *memory coefficient* $m_i = f(\mathbf{x} \cdot \mathbf{k}_i)$ for the $i$-th memory cell. Finally, the output of FFN is the the sum of its values $\mathbf{v}_i$ weighted by their corresponding memory coefficient $m_i$. Conventionally, the *size* of FFN — $d_m$ — is set to be $4 \cdot d$.

### 2.2 Scaling up FFN

As discussed in §1, scaling up the parameter in FFN serves as a lever to improve transformer performance. Since a conventional FFN already takes about two thirds of a transformer layer's parameters (Geva et al., 2021), scaling up FFN will greatly affects the parameter size of a transformer model. However, one could sparsely activate the parameter to control the required compute. In this section, we will review two lines of work to achieve a *scaled* and *sparse* FFN (S-FFN). One by sparsifying a mixture-of-expert model(§2.2.1), and the other by sparsifying a memory model (§2.2.2).

#### 2.2.1 MIXTURE OF EXPERTS (MOE)

Mixture of experts (MoE; Jacobs et al. (1991)) consists of a set of expert models $\{\mathbf{f}_i(\mathbf{x})\}_{i=0}^{B-1}$ and a gating function $\mathbf{g} : \mathbb{R}^d \to \mathbb{R}^B$ to estimates the importance of each expert. Finally, the output is the sum of experts' output weighted by the gate's estimation of weight for that particular expert.

$$\text{MoE}(\mathbf{x}) = \sum_{i \in \mathcal{E} = \{0,1,\cdots,B-1\}} \mathbf{g}_i(\mathbf{x}) \cdot \mathbf{f}_i(\mathbf{x}) \tag{3}$$

Recent work (Du et al., 2021; Lepikhin et al., 2021; Roller et al., 2021; Lewis et al., 2021; Zhou et al., 2022) adopt it to transformer by treating an FFN as one expert and **S**parsely activating the **MoE** (**SMoE**). In SMoE, the gating function (or "router") routes an input token $\mathbf{x}$ to a subset (e.g. 1 or 2) of experts — $\mathcal{E} = \text{subset}(\mathbf{g}(\mathbf{x}))$. Conventionally, SMoE enforces **load balancing** constraints to avoid overly using a few experts while under-utilizing others, and converging to local optima (Shazeer et al., 2017; Eigen et al., 2014). Various SMoEs mainly have two types of gate.

**Learned gate** is parameterized by a set of learnable expert embeddings $\boldsymbol{\theta} = [\mathbf{e}_0; \cdots; \mathbf{e}_{B-1}] \in \mathbb{R}^{B \times d}$, corresponding to each experts. The importance of the $i$-th expert is obtained by $\mathbf{g}_i(\mathbf{x}) = \frac{\exp(\mathbf{e}_i \cdot \mathbf{x})}{\sum_j \exp(\mathbf{e}_j \cdot \mathbf{x})}$. To enforce load balancing when routing, SMoEs use an additional auxiliary loss (Lepikhin et al., 2021; Fedus et al., 2021; Artetxe et al., 2021; Du et al., 2021; Chi et al., 2022) or frame expert utilization as a constrained optimization problem (Lewis et al., 2021; Zhou et al., 2022).

**Static gate**, in contrast to a learnable gate, doesn't have any differentiable parameters. Instead, it uses a static mapping that encodes load-balancing constraints to route input (Roller et al., 2021; Gururangan et al., 2021). For example, RandHash from HashLayer (Roller et al., 2021) uses a hash table that maps from token type to *randomly* selected expert(s) and enforcing the load balancing with random selection. DEMix (Gururangan et al., 2021) ensures $i$-th expert only sees data from $i$-th domain; and each expert is trained for equal amount of steps to ensure equal utilization.

#### 2.2.2 SPARSE NEURAL MEMORY

The other line of work follows the memory view of FFN (Eq. 2). It is straightforward to increase the memory size $d_m$ to a much larger value $d_m \gg 4 \cdot d$. By only using the top-$k$ entries in $\mathbf{m}$, one could sparsely activate the value table, resulting in a vanilla sparse memory (**VanillaM**). The straightforward application, however, uses the entire key table to produce all the memory coefficients $\mathbf{m} = \mathbf{x} \cdot \mathbf{K}^\top$, resulting in dense computation proportional to the memory size. Memory models need to use models that do not look at the full key table to scale computation sublinearly to the number of memory cells and take advantage of sparse computation. Lample et al. (2019) explored the following two techniques in this direction.

**Low-Rank Key Memory (LoRKM)** A straightforward technique is to assume that the key table is composed of and approximated by a down-projection $\mathbf{D} \in \mathbb{R}^{d \times d_\ell}$ and a low-rank key table $\tilde{\mathbf{K}} \in \mathbb{R}^{d_m \times d_\ell}$, where $d_\ell \ll d$. LoRKM produces memory coefficients by $m_i = f((\mathbf{x} \cdot \mathbf{D}) \cdot \tilde{\mathbf{k}}_i) = f(\mathbf{t} \cdot \tilde{\mathbf{k}}_i)$.

**Product Key Memory (PKM)** Building upon LoRKM, PKM assumes that the low-rank key table is further approximated by the outer *product*, w.r.t. vector concatenation operator, of two smaller *sub-key* tables $\mathbf{C}, \mathbf{C}' \in \mathbb{R}^{\sqrt{d_m} \times \frac{d_\ell}{2}}$. Unlike LoRKM where key vectors are independent of each other, PKM has them entangled with one another, i.e. $\tilde{\mathbf{k}}_i = \left[\mathbf{c}_{\lfloor i/\sqrt{d_m} \rfloor}, \mathbf{c}'_{i \pmod{\sqrt{d_m}}}\right] \in \mathbb{R}^{d_\ell}$. One could exploit this structure to efficiently compute the memory coefficient $\mathbf{m}$: calculates the dot product between sub-key tables and down-projected input $\mathbf{t}$ individually and combines them together with a negligible cost to form the full dot product $\mathbf{m} \in \mathbb{R}^{d_m}$:

$$m_i = f\left(\mathbf{s}_{\lfloor i/\sqrt{d_m} \rfloor} + \mathbf{s}'_{i \pmod{\sqrt{d_m}}}\right), \text{ where } \mathbf{s} = \mathbf{t}\left[: \frac{d_\ell}{2}\right] \cdot \mathbf{C}^\top, \mathbf{s}' = \mathbf{t}\left[\frac{d_\ell}{2} :\right] \cdot (\mathbf{C}')^\top \in \mathbb{R}^{\sqrt{d_m}}$$

Since this factorization leads to minuscule $\tilde{\mathbf{K}}(< 1\%)$, one could have multiple randomly initialized key tables to process batched input together and encourage load balancing — uniform usage of $\mathbf{V}$.

# 3 A UNIFIED VIEW OF SPARSE FFNS

On the surface sparse memory and MoE are different, but MoE fits neatly in the memory view. We first derive an equivalent form of MoE to establish its connection with sparse memory and FFN (§3.1). Then, we propose a unified framework for S-FFN (§3.2).

## 3.1 A CLOSER LOOK AT MOE

MoE uses gating function to estimate the importance of all experts and combines the output of each expert mode through linear combination. Here, inspired by the memory view on FFN (§2), we derive from the standard MoE formulation (Eq. 3) and provide an memory view of current MoE:

$$\begin{aligned}
\text{MoE}(\mathbf{x}) &= \sum_{i=0}^{B-1} \mathbf{g}_i(\mathbf{x}) \cdot \mathbf{f}_i(\mathbf{x}) = \sum_{i=0}^{B-1} \mathbf{g}_i(\mathbf{x}) \cdot \text{FFN}^{(i)}(\mathbf{x}) \quad \text{(Each expert is an FFN)} \\
&= \sum_{i=0}^{B-1} \mathbf{g}_i(\mathbf{x}) \cdot \left( \sum_{j=0}^{d_m-1} m_j^{(i)} \cdot \mathbf{v}_j^{(i)} \right) \quad \text{(Apply the memory view of FFN (Eq. 2))} \\
&= \sum_{i=0}^{B-1} \sum_{j=0}^{d_m-1} \left( \mathbf{g}_i(\mathbf{x}) \cdot m_j^{(i)} \right) \cdot \mathbf{v}_j^{(i)} \quad \text{(Distributive property of multiplication)} \qquad (4) \\
&= \sum_{l=0:\ l=i\cdot d_m+j}^{B\cdot d_m-1} \left( \mathbf{g}_i(\mathbf{x}) \cdot m_j^{(i)} \right) \cdot \mathbf{v}_j^{(i)} \ \text{(Re-indexing)} = \sum_{l=0}^{B\cdot d_m-1} m_l \cdot \mathbf{v}_l \qquad (5)
\end{aligned}$$

In Eq 5, $\mathbf{v}_l$ is the $l$-th row of the stack of $d_m$-size value tables from $B$ FFN experts; and $ml = m_{i\cdot d_m+j}$ is its memory coefficient obtained from $\mathbf{g}_i(\mathbf{x}) \cdot m_j^{(i)}$. For example, $m_{d_m}$ is is obtained by weighting $m_0^{(1)}$ — the 0-th memory coefficient in the 1-th FFN expert — with $\mathbf{g}_0(\mathbf{x})$ — the estimation of the 1-th FFN from gate. Building upon such memory view of MoE, one could see that SMoE is a sparse memory operating in terms block and uses its gate to narrow down the summation over the stacked value tables $\mathbf{V}$ to ones in $\text{FFN}^{(i)}$, for $i \in \text{subset}(\mathbf{g}(\mathbf{x})) = \mathcal{E}$.

**Comparison with Sparse Memory** Both SMoE and Sparse Memory can be viewed as neural memory, but there are several differences: **1)** Memory cells share importance weights: in sparse memory, each memory cells receives an individual weight. In contrast, in SMoE groups of $4 \cdot d$ share the same importance weight $\mathbf{g}_i(\mathbf{x})$. **2)** Single selection and importance weight: Sparse memory uses a single dot product between input token and any key vector $\mathbf{x} \cdot \mathbf{k}$ for both, whereas SMoE depends on a separately parameterized gate $\mathbf{g}$ in addition to the dot product.

## 3.2 THE UNIFIED FRAMEWORK

We propose a general framework that unifies the view of different S-FFN. We cast both as instances of a memory with large key and value table — $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{d_m \times d}$, where $d_m \gg 4 \cdot d$. We distinguish the different methods among two dimensions illustrated below and summarized in Fig. 2:

**Memory block size** specified how many memory cells share the same importance weight at selection time, and thus together treated as a memory block. We use $g$ to denote the size of one block. In another word, we split the $\mathbf{K}, \mathbf{V}$ along the $d_m$-dimension into $g$-size blocks. Therefore, a memory consists of $B = d_m/g$ blocks in total. Formally, we write

$$\mathbf{K}^g = \left[ \mathbf{K}^{(0)}; \mathbf{K}^{(1)}; \cdots; \mathbf{K}^{(B-1)} \right] \in \mathbb{R}^{d_m \times d_k}, \mathbf{V}^g = \left[ \mathbf{V}^{(0)}; \mathbf{V}^{(1)}; \cdots; \mathbf{V}^{(B-1)} \right] \in \mathbb{R}^{d_m \times d_v}$$

For example, sparse memory has block size $g = 1$ — trivially treating 1 memory cell as a "block"; and MoE has the block size $g = 4 \cdot d$ (§3.1). Current approaches have been generally tested with fixed block sizes, but this is mostly an artifact of how the methods were derived rather that a mathematical constraint. For example, we can design SMoE versions that instead of 1 expert of size $4 \cdot d$, or alternatively, uses 2 experts of size $2 \cdot d$. We can similarly chunk memory coefficients $\mathbf{m}$ into blocks of size $g$ when dealing with sparse memories.

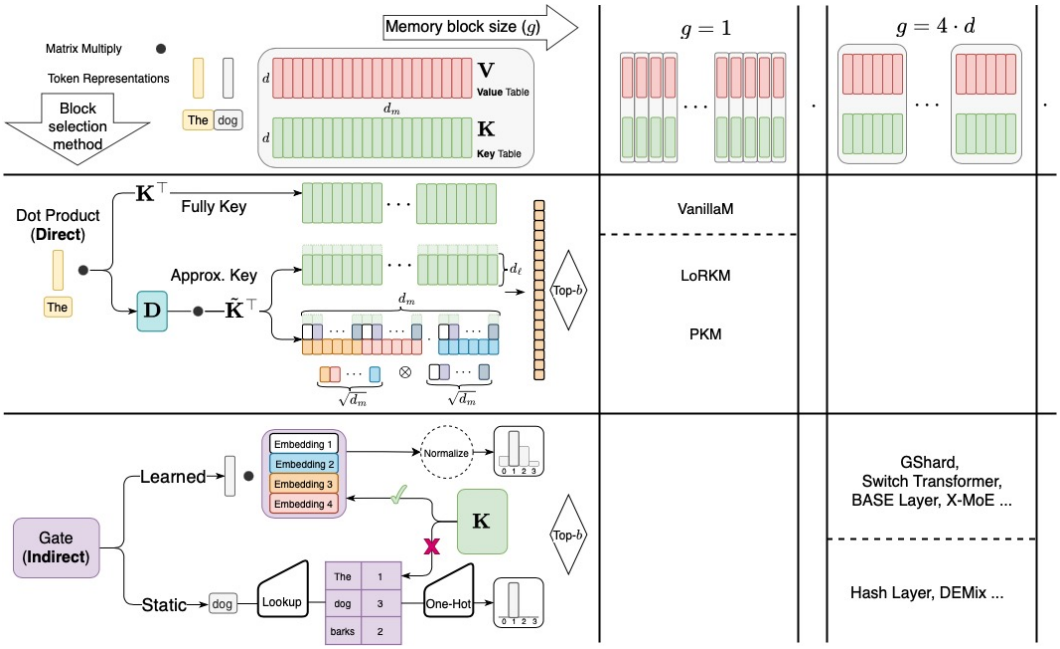



Figure 2: S-FFN methods decomposed along the defined design dimensions.

**Block selection method** is the specific function that compute the importance of each memory blocks for selection. Since SMoE is also a type of sparse memory, we distinguish selection method by a new criterion — *whether one allows input* $\mathbf{x}$ *to directly interact with the key table* $\mathbf{K}^g$. As discussed in §3.1, SMoE uses the estimation from an individually parameterized gate to select, while sparse memory solely and directly uses key table. Thus, current SMoE is a type of *indirect* selection method, and sparse memory a *direct* one. Various SMoE is further characterized by whether its gating function has learned parameter or consists of a static mapping (§2.2.1). On the other hand, sparse memory is characterized by how much factorization is applied to the key table (§2.2.2).

**Load balancing as an external factor** Training a language model at least comprise of three components: model, objective loss, and data. In this work, we only focus on the *model* aspect. Usually, the load balancing is usually related to non-model aspect (§2.2.1). Therefore, we treat the load balancing as an external factor and exclude an explicit study on load balancing.

We want to note there are many unexplored spaces in Fig. 2. However, it is unrealistic to "grid search" the table in a pretraining setting. Critically, the unified framework allows us to study S-FFN methods along these design dimensions to identify the critical design choices (§5). Consequentially, we propose a new method — **`Avg-K`**, combining the advantages of direct and indirect methods. (§6).

# 4 Experiment Setup

## 4.1 Models

In this work, we follow GPT-3 (Brown et al., 2020) to train several transformer models. Our chosen architecture has 24 transformer layers, with $d = 1024$, GeLU activation functions and set memory size (or hidden size) to be $4 \cdot d$. We denote this model as **Dense Baseline**.

**S-FFN** Given a model above, we replace some of its FFNs with an S-FFN. To finish training within a reasonable time frame , we replace FFN every 6 layers, leading to 4 S-FFN in total. To control how activated the S-FFN is, we use $k$ to denote the number of active memory cells, and the formulation of $d_m = E \cdot (4 \cdot d)$ to control the size of S-FFN. Therefore, with block size $g$, the S-FFN will activate $b = \frac{k}{g}$ among $B = \frac{d_m}{g}$ memory blocks. In Table 1, we lists all experiments used for analysis in §5.

## 4.2 Language Modeling

**Pretraining** We pretrain our models on a union of six English-only datasets, including the five datasets used to pretrain RoBERTa (Liu et al., 2019) and the English subset of CC100, to-

| Method type | Method name | $g$ | $E$ | $k$ |
|---|---|---|---|---|
| Direct | VanillaM | $\{1, 64, 256, 1024, 2048, 4096\}$ (§5.1) | $\{4, 16, (32)^*\}$ | $\{4096, (8192)^*\}$ |
| | LoRKM | $\{1\}$ | | |
| | PKM (Lample et al., 2019) | $\{1\}$ | | |
| Indirect | RandHash (Roller et al., 2021) | $\{1, 64, 256, 1024, 2048, 4096\}$ (§5.1) | | |
| | Switch (Fedus et al., 2021) | $\{4096\}$ | | |
| | PKM-FFN (§5.2) | $\{1\}$ | | |

Table 1: All the experiments used for analysis in §5. Some settings(*) are only used for PKM.

talling 112B tokens corresponding to 453GB — BookCorpus (Zhu et al., 2015), English Wikipedia, CC-News (Nagel, 2016), OpenWebText (Gokaslan & Cohen, 2019), CC-Stories (Trinh & Le, 2018) English CC100 (Wenzek et al., 2020) (details in Appendix A.2) We encode our data using the same Byte-Pair Encoding as GPT-2 (Radford et al., 2019) and RoBERTa (Liu et al., 2019) with a vocabulary of 50K subword units. Due to limited budget, we set the total number of training tokens to 60B for a reasonable training time. We left the hyperparameters at Appendix A.1.

**Evaluation settings** We evaluate our models on their ability to predict the next token in a sequence as measured by perplexity (the lower the better). Other than **in-domain**(train and validation), we also report perplexity in **out-of-domain settings** to indicate generalization ability. We sample a held-out subset of the combined pretraining data (§4.2) for in-domain evaluation. For out-of-domain, we use data from The Pile Gao et al. (2020), a public dataset that combines data from 22 diverse sources.

## 5 Analysis with our unified view

In this section, we use our proposed unified view to systematically study the design choice of S-FFN. Specifically, **1)**. the current choice of block size seems incidental, and how it affects the perplexity requires careful study (§5.1). **2)**. Both direct and indirect methods lead to better performance than a standard FFN, but which method has better FLOPs-Perplexity trade-off and what are the relative efficacy of different methods requires further studies (§5.2).

### 5.1 Memory block size

Since block size is a natural number, we aim to answer a straightforward question — **given a fixed $k$, does smaller memory block size lead to lower perplexity?** For a comprehensive study and to avoid the impact from hyperparameters, we uses HashLayer(Roller et al., 2021) with random hash (denoted **RandHash**) for indirect, and VanillaM for direct selection. For all experiments, we use $E = 16$. In the following, we first describe how we modify both methods to support different block sizes.

**RandHash** randomly selects $b = \frac{k}{g}$ unique experts among all $B = \frac{d_m}{g}$ experts — essentially sampling $b$ unique values from Uniform($[0, \cdots, B-1]$). Originally, with $g = 4096$, a RandHash assigns a token to $\frac{4096}{4096} = 1$ expert; with $g = 2048$, $\frac{4096}{2048} = 2$ experts.

**VanillaM** originally has a block size $g = 1$ and selects top-$k$ scalars in memory coefficients $\mathbf{m} = \text{GeLU}(\mathbf{x} \cdot \mathbf{K}^\top)$. We made a *minimal change* to extend it to larger $g$: given $\mathbf{m}$, we chunk it into $B$ blocks — $\mathbf{m}^g = [\mathbf{m}^{(0)}; \cdots; \mathbf{m}^{(B-1)}]$; then, we select the top-$b$ blocks using the average of each block — $\texttt{Avg}(\text{GeLU}(\mathbf{x} \cdot (\mathbf{K}^{(i)})^\top), \texttt{dim=0})$.[1]

In Fig. 3, we consistently observe that smaller block size leads to around $-0.7$ perplexity across the two methods. This observation holds no matter how one selects: **1)** with indirect (RandHash) or direct (VanillaM) selection and **2)** with random (RandHash) or similarity-based (VanillaM) selection. In Appendix B, we show the that smaller block size allows larger flexibility of more combination of memory cells. For example, when $g$ is halved, a half memory cells of expert 1 could be activated together with a half of expert 2; however, this combination is not possible with a larger $g$.

**Expenses of smaller $g$ for indirect method** In a data parallelism setting, memory blocks are also distributed across multiple GPUs and the memory calculation are parallelized (model parallelism).

[1] $\texttt{Avg}(\cdot)$ wins over other simple aggregators — $\texttt{Min}(\cdot)$, $\texttt{Max}(\cdot)$, and $\texttt{Avg}(|\cdot|)$; see details in Appendix.

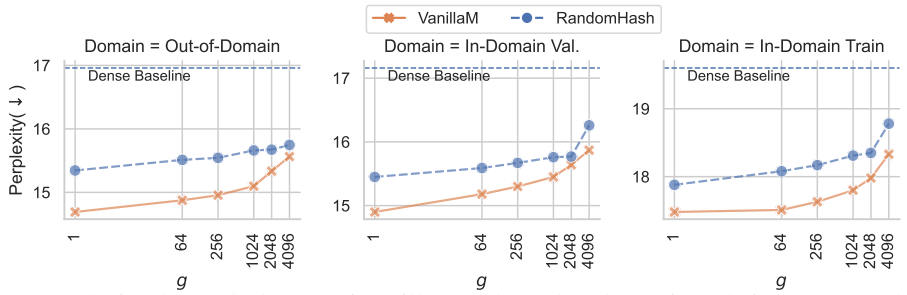



Figure 3: Perplexity (lower the better) of VanillaM and RandHash consistently improve as $g$ decreases.

Conceptually, for example, with $g = 4 \cdot d$, a token is only routed to 1 memory block on one device; and, if $g = 2 \cdot d$, each device double its chance to receive more tokens. Although communication cost remain the same since no devices are added, each GPU needs to process more tokens and leads to more compute time. In addition, since each memory block has its representation in gate, smaller block will lead to more block representation stored in gate, e.g. more learned parameters in learned gate, a larger table for static gate. Accordingly, there will be more computes in computing $\mathbf{g}(\mathbf{x})$.

### 5.2 Block Selection Method

Having concluded that smaller $g$ leads to smaller perplexity, one might expect current direct methods ($g = 1$) will outperform indirect ones ($g = 4 \cdot d$). However, on the other side, efficient direct methods uses factorized, and thus less expressive keys, while indirect methods uses unfactorized keys. Additionally, both methods outperforms Dense Baseline, so it remains unclear and interesting to understand which methods has better FLOPs-Perplexity trade-off. Then, we devise some contrastive pair of model designs to get more insights on relative efficacy.

**FLOPs-Perplexity trade-off (Pareto Efficiency)** In this study, we first try to answer a straightforward question — **which, direct or indirect, method could decrease more perplexity with little additional FLOPs**, and thus a better FLOPs-perplexity trade-off? To create a scaling scenario, we change the size of the memory by varying $E \in \{4, 16\}$; and additionally run $E = 32$ for PKM.

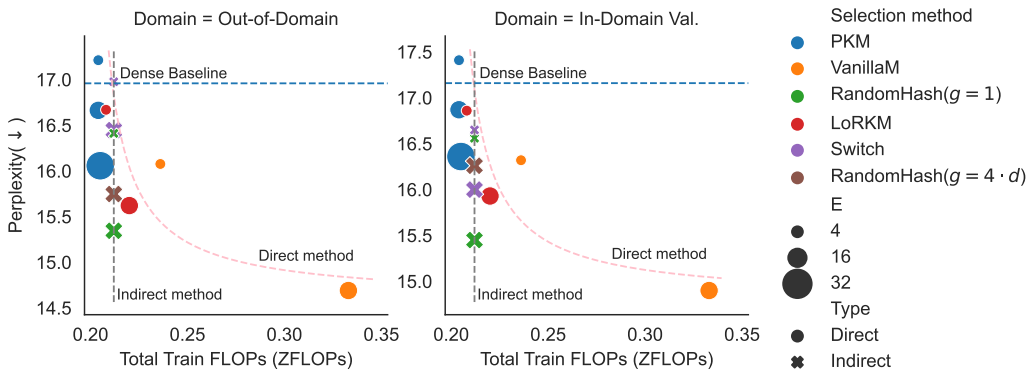



Figure 4: Pareto Frontier of different models. Indirect method has the same FLOPs cost as Dense Baseline. We manually draw illustrative curves for each method to demonstrate their scaling behavior — how much perplexity improvement w.r.t. additional FLOPs. The steep curve of indirect method implies its scaling advantage over current sparse memory.

As shown in Fig. 4, indirect selection (i.e. SMoE) of various memory sizes has essentially the same training FLOPs as our Dense Baseline. In contrast, the perplexity of all direct methods is inversely proportional to its training FLOPs. Thus, indirect methods are more Pareto-efficient. Since sparse memories are trained with $g = 1$, so they are at their best perplexity (§5.1). Noticeably, PKM is the best scaling choice among all direct methods, because the training cost stays similar. Since the down-projection and key table in PKM $^{E=32}$ only takes up $0.13\%$, we double its $k$ to make fair comparison with indirect methods with $E$. However, we surprisingly observe an increase in perplexity (Table 2).

In the following paragraphs, we will present several contrastive pair of direct & indirect methods and learn important lessons for critical aspects for a better design. To simplify notation, if unspecified, we assume $E = 16$ and $g = 1$.

**Finding 1: A fully parameterized key table is required** We note that PKM and RandHash have the same value table, but the key table of PKM is minuscule w.r.t. that of RandHash. By considering the "edit distance" of PKM $^{E=32}$ and RandHash from PKM, we realize that their only difference, other than selection method, is where the additional parameter is added: RandHash use it for a fully parameterized key table, but PKM $^{E=32}$ trade it for doubling the value table. Inspired by such comparison, we introduce a compute-efficient learned gate to replace RandHash's gate for analytial purpose. The output of this gate is the same as the $\mathbf{m}$ from a PKM and we uses $\mathbf{g}_i = m_i$ to avoid the compute-heavy Softmax. We call it **PKM-FFN**. As shown in Table 2, on out-of-domain, PKM-FFN substantially outperform ($-1$ perplexity) parameter- and compute-matched PKM $^{E=32}_{k=8192}$ and slightly outperform RandHash by about 0.16, which only differs by selection methods. Therefore, we conclude that having an efficient gate doesn't affect performance at $g = 1$, but it is **essential to have a full-parameter, and thus expressive enough, key table to produce memory coefficients**.

| Method type | Name | $E$ | #Params (Entire Model) | Train ZFLOPs | Out-of-Domain (22 domains; Avg. $\pm$ Std.) | In-Domain Train | In-Domain Val. |
|---|---|---|---|---|---|---|---|
| | Dense Baseline | 1 | 354.7M | 0.212 | $16.96 \pm 5.20$ | 19.60 | 17.16 |
| Direct | PKM ($k = 4096$) | 32 | 858.7M | 0.205 | $16.06 \pm 4.84$ | 18.93 | 16.36 |
| | PKM ($k = 8192$) | 32 | 858.7M | 0.213 | $16.16 \pm 4.88$ | 19.05 | 16.45 |
| | VanillaM | 16 | 858.3M | 0.333 | $14.69 \pm 4.31$ | 17.48 | 14.90 |
| Indirect | PKM-FFN | 16 | 858.9M | 0.213 | $15.19 \pm 4.54$ | 17.82 | 15.48 |
| | RandHash | 16 | 858.3M | 0.212 | $15.35 \pm 4.59$ | 17.88 | 15.45 |

Table 2: Experiments for contrastive comparison for model design. All models have $g = 1$.

**Finding 2: Use dot product $\mathbf{x} \cdot (\mathbf{K}^g)^\top$ information to select** Table 2 shows the improvement of VanillaM, PKM-FFN, RandHash over Dense Baseline on out-of-domain are 2.27, 1.77, and 1.61 respectively on out-of-domain. They all have the same key and value table, but only differ by how much the dot product information is used for selection — VanillaM directly uses it, PKM-FFN uses it indirectly when the gate is jointly trained, and RandHash completely ignore it. Therefore, we posit that **the more dependent on dot product the selection method is, the better language model it will leads to and indirect usage (PKM-FFN) is not enough.**

## 6 A NEW ROUTING METHOD — `Avg-K`

Based on analysis insights from above, an optimal gating function need to be: with unfactorized $\mathbf{K}^g, \mathbf{V}^g$, it allows operation or equivalent one to *directly* use dot product between $\mathbf{x}$ and $\mathbf{K}^g$, and doesn't incur too much cost with smaller $g$. Towards this end, we propose a new routing methods — `Avg-K`. We represent each block with the average of its key table $\mathbf{K}^{(i)} \in \mathbb{R}^{g \times d}$ along $g$-dimension — $\mathbf{e}_i = \frac{1}{g}\sum_{j=0}^{g-1} \mathbf{k}_j^{(i)} = \texttt{Avg}(\mathbf{K}^{(i)}, \texttt{dim=0})$. Then, we use the dot product between $\mathbf{x}$ and the averages to select the top-$b$ selected block and route the token there for memory calculation (Eq. 2):

$$\mathbf{g}_i(\mathbf{x}) = \begin{cases} 1 & i \in \mathcal{E} = \{\text{top-}b \text{ of } [\mathbf{e}_0 \cdot \mathbf{x}, \cdots, \mathbf{e}_{B-1} \cdot \mathbf{x}]\} \\ 0 & \text{otherwise} \end{cases}$$

Due to the linearity of averaging, the operation $\mathbf{e}_i \cdot \mathbf{x}$ is equivalent to calculate the average of dot products within a block without GeLU. Since all tokens share the averages, our method is efficient.

Table 3 shows our methods is the best among all indirect methods. With $< 1\%$ additional FLOPs, `Avg-K` achieves 2.16 lower perplexity than a vanilla transformer, while Switch Transformer 0.51 lower and the best RandHash reaches 1.61 lower. Due to the similar form as VanillaM, in Appendix C.1, we include contrastive analysis to understand the drastic improvement from $g = 4096$ to 256.

**Discussion** `Avg-K` essentially applies an average pooling to the unfactorized $\mathbf{K}^g$. We believe this is a better way to create block representations and leads to better $\mathbf{K}^g, \mathbf{V}^g$. Without load balancing

| $E$ | #Params (Entire Model) | Selection method | $g$ | Train ZFLOPs | Out-of-Domain (22 domains; Avg. $\pm$ Std.) | In-Domain Train | In-Domain Val. |
|---|---|---|---|---|---|---|---|
| 1 | 354.7M | Dense Baseline | 1 | 0.212 | $16.96 \pm 5.20$ | 19.60 | 17.16 |
| 16 | 858.3M | RandHash | 4096 | 0.212 | $15.75 \pm 4.73$ | 18.78 | 16.26 |
| | | | 1 | 0.212 | $15.35 \pm 4.59$ | 17.88 | 15.45 |
| | | PKM-FFN | 1 | 0.213 | $15.19 \pm 4.54$ | 17.82 | 15.48 |
| | | Switch | 4096 | 0.212 | $16.45 \pm 5.06$ | 18.20 | 16.00 |
| | | **Avg-K** | 4096 | 0.212 | $16.44 \pm 4.98$ | 19.04 | 16.59 |
| | | | 2048 | 0.212 | — | 18.60 | 16.19 |
| | | | 1024 | 0.212 | — | 18.14 | 15.73 |
| | | | 256 | 0.213 | $14.91 \pm 4.40$ | 17.57 | 15.19 |
| | | | 64 | 0.214 | $14.80 \pm 4.39$ | 17.51 | 15.11 |
| | | VanillaM | 1 | 0.33 | $14.69 \pm 4.31$ | 17.48 | 14.90 |

Table 3: Comparing **Avg-K** with other indirect methods.

constraints, SMoE with learned gate will degenerate in using only a handful of memory blocks (§2.2.1), and **Avg-K** also uses blocks disproportionally (Appendix C.2). However, **Avg-K** still matches the performance of Switch (i.e. $g = 4096$) or outperform all other SMoE (i.e. $g \leq 256$). Nevertheless, new research questions are open to discussion: how adding the constraint will effect the performance, whether `Avg` is the best function to "pool" $\mathbf{K}$, or whether dot product is the best metric for selection.

Previously, Geva et al. (2021) shows that rows of $\mathbf{K}$ corresponds (i.e. by dot products) to different — both surface- and semantic-level — textual features. Since **Avg-K** heavily depends on dot products, their analysis could be easily extended for interpreting our model. In comparison, RandHash is a lot harder for interpretation due to random selection, and the relationship between $\mathbf{e}_i$ and $\mathbf{K}^{(i)}$ remains unclear for Switch or SMoE with learned gate. We defer such analysis to future work, since this is beyond the scope of our paper.

**Limitation** In large-scale SMoE training, the speed is limited by the most heavy-loaded GPU when model parallelism is used. Therefore, load balancing is important. Since our scale is relatively small and we didn't uses model parallelism as a preliminary study the problem is not so pronounced for us.

## 7 Related Work

**Excluded S-FFN** Terraformer's Jaszczur et al. (2021) technique on FFN is closest to our PKM-FFN because there's a low-rank learned gate to score each memory cells of $\mathbf{K}, \mathbf{V}$ for selection. However, this method is excluded because our framework uses all memory cells in each block, but Terraformer selects 1 cell in each memory block. In Appendix D.1, we study an VanillaM version of their method and found their "one-per-block" design limits the model's performance.

**Approximate Nearest Neighbour(ANN) search** One might wonder whether ANN techniques could help searching best key in VanillaM rather than trade the expressiveness of key table for efficiency. For example, one could process the unfactorized key table by ANN methods like FAISS (Johnson et al., 2021), ScaNN (Guo et al., 2020) or FALCONN (Andoni et al., 2015). One of successful example is applying vanilla Locality-Sensitive Hashing to Reformer (Kitaev et al., 2020). However, in our preliminary study, we found perplexity is greatly affected by the quality of search and building data structure after every update is expensive and hard to avoid. We leave detailed discussion and analysis to Appendix D.2.

## 8 Conclusion

We provide a unified framework for existing S-FFN methods that are seemingly different. This framework enables us to instantiate a simpler and potentially more interpretable model that achieves better performance by producing better representation. We hope our work can provide insights and guidance for future research on S-FFN.

## REFERENCES

Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya P. Razenshteyn, and Ludwig Schmidt. Practical and optimal LSH for angular distance. In Corinna Cortes, Neil D. Lawrence, Daniel D. Lee, Masashi Sugiyama, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, pp. 1225–1233, 2015. URL https://proceedings.neurips.cc/paper/2015/hash/2823f4797102ce1a1aec05359cc16dd9-Abstract.html.

Mikel Artetxe, Shruti Bhosale, Naman Goyal, Todor Mihaylov, Myle Ott, Sam Shleifer, Xi Victoria Lin, Jingfei Du, Srinivasan Iyer, Ramakanth Pasunuru, Giri Anantharaman, Xian Li, Shuohui Chen, Halil Akin, Mandeep Baines, Louis Martin, Xing Zhou, Punit Singh Koura, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Mona T. Diab, Zornitsa Kozareva, and Ves Stoyanov. Efficient large scale language modeling with mixtures of experts. *CoRR*, abs/2112.10684, 2021. URL https://arxiv.org/abs/2112.10684.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

Zewen Chi, Li Dong, Shaohan Huang, Damai Dai, Shuming Ma, Barun Patra, Saksham Singhal, Payal Bajaj, Xia Song, and Furu Wei. On the representation collapse of sparse mixture of experts. *CoRR*, abs/2204.09179, 2022. doi: 10.48550/arXiv.2204.09179. URL https://doi.org/10.48550/arXiv.2204.09179.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *CoRR*, abs/2204.02311, 2022. doi: 10.48550/arXiv.2204.02311. URL https://doi.org/10.48550/arXiv.2204.02311.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-1423. URL https://doi.org/10.18653/v1/n19-1423.

Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathy Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen,

and Claire Cui. Glam: Efficient scaling of language models with mixture-of-experts. *CoRR*, abs/2112.06905, 2021. URL https://arxiv.org/abs/2112.06905.

David Eigen, Marc'Aurelio Ranzato, and Ilya Sutskever. Learning factored representations in a deep mixture of experts. In Yoshua Bengio and Yann LeCun (eds.), *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*, 2014. URL http://arxiv.org/abs/1312.4314.

William Fedus, Barret Zoph, and Noam M. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *ArXiv*, abs/2101.03961, 2021.

William Fedus, Jeff Dean, and Barret Zoph. A review of sparse expert models in deep learning, 2022. URL https://arxiv.org/abs/2209.01667.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pp. 5484–5495. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.446. URL https://doi.org/10.18653/v1/2021.emnlp-main.446.

Aaron Gokaslan and Vanya Cohen. Openwebtext corpus. *http://web.archive.org/save/http://skylion007.github.io/OpenWebTextCorpus*, 2019.

Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*, 2020. URL https://arxiv.org/abs/1908.10396.

Suchin Gururangan, Mike Lewis, Ari Holtzman, Noah A. Smith, and Luke Zettlemoyer. Demix layers: Disentangling domains for modular language modeling. *CoRR*, abs/2108.05036, 2021. URL https://arxiv.org/abs/2108.05036.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *CoRR*, abs/2203.15556, 2022. doi: 10.48550/arXiv.2203.15556. URL https://doi.org/10.48550/arXiv.2203.15556.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3:79–87, 1991.

Sebastian Jaszczur, Aakanksha Chowdhery, Afroz Mohiuddin, Lukasz Kaiser, Wojciech Gajewski, Henryk Michalewski, and Jonni Kanerva. Sparse is enough in scaling transformers. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pp. 9895–9907, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/51f15efdd170e6043fa02a74882f0470-Abstract.html.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Trans. Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572. URL https://doi.org/10.1109/TBDATA.2019.2921572.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL https://openreview.net/forum?id=rkgNKkHtvB.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 8546–8557, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/9d8df73a3cfbf3c5b47bc9b50f214aff-Abstract.html.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam M. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *ArXiv*, abs/2006.16668, 2021.

Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. BASE layers: Simplifying training of large, sparse models. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 6265–6274. PMLR, 2021. URL http://proceedings.mlr.press/v139/lewis21a.html.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. 2019. URL http://arxiv.org/abs/1907.11692.

Sebastian Nagel. Cc-news. *http://web.archive.org/save/http://commoncrawl.org/2016/10/news-dataset-available*, 2016.

Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models. *CoRR*, abs/2106.04426, 2021. URL https://arxiv.org/abs/2106.04426.

Roy Schwartz, Jesse Dodge, Noah Smith, and Oren Etzioni. Green ai. *Communications of the ACM*, 63:54 – 63, 2020.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL https://openreview.net/forum?id=B1ckMDqlg.

Sainbayar Sukhbaatar, Arthur D. Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *NIPS*, 2015a.

Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015b.

Sainbayar Sukhbaatar, Edouard Grave, Guillaume Lample, Herve Jegou, and Armand Joulin. Augmenting self-attention with persistent memory. *arXiv preprint arXiv:1907.01470*, 2019.

Trieu H. Trinh and Quoc V. Le. A simple method for commonsense reasoning. *ArXiv*, abs/1806.02847, 2018.

Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *ArXiv*, abs/1706.03762, 2017.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *CoRR*, abs/2206.07682, 2022. doi: 10.48550/arXiv.2206.07682. URL https://doi.org/10.48550/arXiv.2206.07682.

Guillaume Wenzek, Marie-Anne Lachaux, Alexis Conneau, Vishrav Chaudhary, Francisco Guzm'an, Armand Joulin, and Edouard Grave. Ccnet: Extracting high quality monolingual datasets from web crawl data. In *LREC*, 2020.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. OPT: open pre-trained transformer language models. *CoRR*, abs/2205.01068, 2022. doi: 10.48550/arXiv.2205.01068. URL https://doi.org/10.48550/arXiv.2205.01068.

Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Y. Zhao, Andrew M. Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. *CoRR*, abs/2202.09368, 2022. URL https://arxiv.org/abs/2202.09368.

Yukun Zhu, Ryan Kiros, Richard S. Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. *2015 IEEE International Conference on Computer Vision (ICCV)*, pp. 19–27, 2015.

# A Training Setting

## A.1 Hyperparameters

Table 5 specifies shared hyperparameters across all experiments, in which Table 4a contains ones for training data, optimizer, and efficient infrastructure techniques; and Table 4b for architecture. Then, Table 5a describes the hyperparameters specifically for Switch, Table 5b for LoRKM, Table 5c for PKM,

## A.2 Data

Here is a detailed description of our pretraining corpus.

- **BookCorpus** (Zhu et al., 2015) consists of more than 10K unpublished books (4GB);
- **English Wikipedia**, excluding lists, tables and headers (12GB);
- **CC-News** (Nagel, 2016) contains 63 millions English news articles crawled between September 2016 and February 2019 (76GB);
- **OpenWebText** (Gokaslan & Cohen, 2019), an open source recreation of the WebText dataset used to train GPT-2 (38GB);
- **CC-Stories** (Trinh & Le, 2018) contains a subset of CommonCrawl data filtered to match the story-like style of Winograd schemas (31GB);
- **English CC100** (Wenzek et al., 2020), a dataset extracted from CommonCrawl snapshots between January 2018 and December 2018, filtered to match the style of Wikipedia (292GB).

# B Block size analysis

We first quantify the intuition —"usage of model memory is more spread out" by number of activated memory cells shared between two random tokens — $\mathbb{E}[r]$. We define this quantity to be average of every S-FFN layer, to reflect the overall behavior — $\mathbb{E}[r] = \frac{1}{L_{\text{S-FFN}}} \sum_\ell \mathbb{E}[r_\ell]$, where $L_{\text{S-FFN}}$ is the

(a) Shared configuration for data, optimizer, and efficient infrastructure

| Name | Values |
|---|---|
| #Tokens for training | 60e9 |
| #Tokens for warmpup | $375 \cdot 1024^2$ |
| #Tokens per batch | $0.5 \cdot 1024^2$ |
| #Tokens per sample | 2048 |
| #GPU | 32 |
| GPU | NVIDIA Tesla V100 32GB |
| Optimizer | Adam($\beta$s $= (0.9, 0.98)$, $\epsilon = 1e-8$) |
| Weight Decay | 0.01 |
| Peak Learning Rate | 3e-4 |
| Learning Rate Scheduler | `polynomial_decay` |
| `clip_norm` | 0.0 |
| DistributedDataParallel backend | FullyShardedDataParallel |
| `memory-efficient-fp16` | True |
| `fp16-init-scale` | 4 |
| `checkpoint-activations` | True |

(b) Shared configuration for architecture.

| Name | Values |
|---|---|
| Objective | Causal Language Model (CLM) |
| Activation function($f$) | GeLU |
| Model dimension ($d$) | 1024 |
| $d_m$ of non-S-FFN | $4 \cdot 1024$ |
| #Attention Head | 16 |
| #Layer | 24 |
| Dropout Rate | 0.0 |
| Attention Dropout Rate | 0.0 |
| `share-decoder-input-output-embed` | True |

Table 4: Shared configuration

(a) Switch

| Name | Values |
|---|---|
| `moe-gating-use-fp32` | True |
| `moe-gate-loss-wt` | 0.01<br>i.e. CLM loss + $0.01\cdot$ auxiliary loss (Fedus et al., 2021) |
| Divide expert gradients by | $\sqrt{\text{\# Expert}} = \sqrt{B}$ |

(b) LoRKM

| Name | Values |
|---|---|
| $d_\ell$ | 128 |
| BatchNorm after $\mathbf{x} \cdot \mathbf{D}$ | False |

(c) PKM

| Name | Values |
|---|---|
| $d_\ell$ | 128 |
| # key table (§2.2.2) | 1 |
| BatchNorm after $\mathbf{x} \cdot \mathbf{D}$ | True |

Table 5: Specific architecture configuration

number of S-FFN. Because block selection usually depends on a contextualized token embedding, it's hard to draw tokens in an i.i.d. fashion. Therefore, we estimate the the quantity by evaluating the model on a validation set. We sample $N$ token pairs from each sequence for estimation:

$$\mathbb{E}[r_\ell] = \frac{1}{|\text{val}| \cdot N} \sum_{s \in \text{val}} \sum_{i=0:(x,y)_i \sim \text{Uniform}(s \times s)}^{N-1} |\mathcal{I}_x \cap \mathcal{I}_y| \cdot g \tag{6}$$

where $\mathcal{I}_x$ is the indices of selected memory block for token at position $x$, and similarly for $\mathcal{I}_y$.

RandHash, though, is an exception where uniform sampling is used. Therefore, $\mathbb{E}[r]$ could also be analytically calculated for various $g$, *when assuming tokens are also uniformly distributed*.

$$\mathbb{E}[r] = \frac{1}{L_{\text{S-FFN}}} \cdot L_{\text{S-FFN}} \cdot \mathbb{E}[r_\ell] = \sum_{i=1}^{b} \underbrace{\binom{b}{i}}_{\substack{\text{No. of such} \\ \text{block assignments}}} \cdot \underbrace{\prod_{j=0}^{i-1} \frac{b-j}{B-j}}_{\substack{\text{Probability of} \\ i \text{ overlaps}}} \cdot \underbrace{\prod_{k=0}^{b-i-1} \frac{B-b-k}{B-k}}_{\substack{\text{Probability of} \\ j \text{ non-overlaps}}} \cdot \underbrace{i \cdot g}_{\substack{r \text{ cells} \\ \text{in an overlap}}} \tag{7}$$

In Fig. 5a, 5b, we evaluate our model with $E = 16$ on our validation subset and calculate the estimations across various $g$. **It is observed that less sharing happens as block size decreases.** However, the empirical estimation for RandHash are relatively constant across granularity. We suspect this is due to the Zipf's law of tokens. Also, we note that the magnitude of $\mathbb{E}[r]$ are different for different methods. We defer the reason of this phenomena to future work.

# C **Avg-K** ANALYSIS

## C.1 **Avg-K** ANALYSIS BY COMPARISON WITH VANILLAM

**Avg-K** essentially applies an average pooling to the unfactorized $\mathbf{K}^g$ to create representation of each block. Due to the linearity of averaging, the operation $\mathbf{e}_i \cdot \mathbf{x}$ is equivalent to calculate the average of dot products within a block before GeLU and select blocks with the average of dot products:

$$\mathbf{e}_i \cdot \mathbf{x} = \left( \frac{1}{g} \cdot \sum_{j=0}^{g-1} \mathbf{k}_j^{(i)} \right) \cdot \mathbf{x} = \frac{1}{g} \cdot \sum_{j=0}^{g-1} \left( \mathbf{k}_j^{(i)} \cdot \mathbf{x} \right) = \texttt{Avg} \left( \mathbf{x} \cdot \left( \mathbf{K}^{(i)} \right)^\top, \texttt{dim=0} \right) \quad (\textbf{Avg-K}) \tag{8}$$

In contrast, VanillaM uses average *after* GeLU(§5.1):

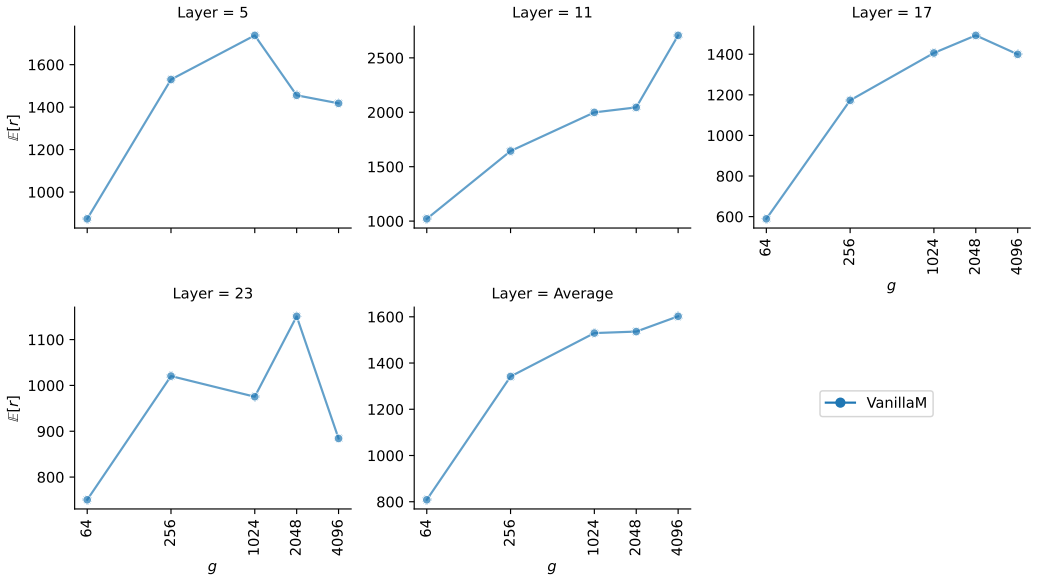
$$\frac{1}{g} \sum_{j=0}^{g-1} \text{GeLU} \left( \mathbf{k}_j^{(i)} \cdot \mathbf{x} \right) \quad (\text{VanillaM})$$

Because GeLU is a non-linear function, average from **Avg-K** could be shared across tokens. In contrast, VanillaM can't, and thus making **Avg-K** efficient.

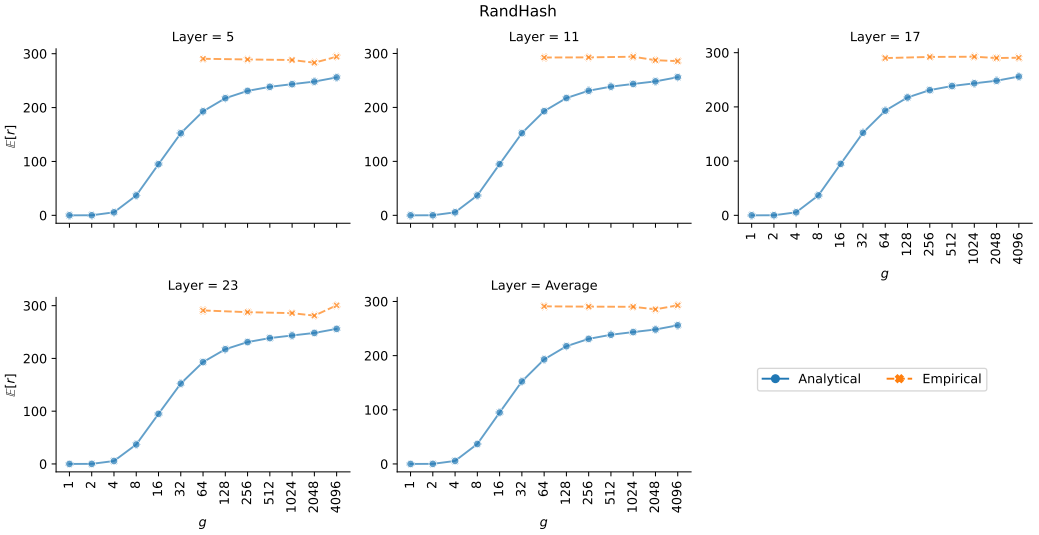
In Fig 6, we experiment both methods with various $g$. We observe when $g$ decreases from $4096$, the perplexity of **Avg-K** drops more drastically than VanillaM. This observation highlights the impact of GeLU. Because $\lim_{x \to -\infty} \text{GeLU}(x) = 0$, it protects the average in VanillaM from some very negative values. Thus, **Avg-K** with larger $g$ included more and potentially very negative values to average over, and thus leads to worse choices than ones made by VanillaM. On the other hand, when $g$ decreases, this "negative value" problem is mitigated. When there are more blocks available for selection (smaller $g$), because negative dot products affects **Avg-K** more, it prefers blocks with more or very positive dot products; whereas, VanillaM is protected from negative value so it fails to detect those blocks. Therefore, **Avg-K** with $g \leq 256$ could achieve an even better perplexity.

## C.2 **Avg-K** LOAD BALANCING

On the same validation set as used in §B, we also conduct a load balancing analysis of memory blocks. Fig. 7 shows that **Avg-K** and VanillaM disproportionally used some memory blocks.




(a) VanillaM: empirical estimation from sampling




(b) RandHash: both analytical value from close form calculation and empirical value from sampling

Figure 5: Expected of shared memory cells across various block size $g$

# D PRELIMINARY STUDY FOR RELATED WORK

## D.1 TERRAFORMER ANALYSIS

**Controller in Terraformer** Jaszczur et al. (2021) uses a controller to score all memory cells and pre-select a subsets — Controller($\mathbf{x}$) — for computation.

$$\mathbf{y} = \sum_{i \in \text{Controller}(\mathbf{x})} f(\mathbf{x} \cdot \mathbf{k}_i) \cdot \mathbf{v}_i \tag{9}$$

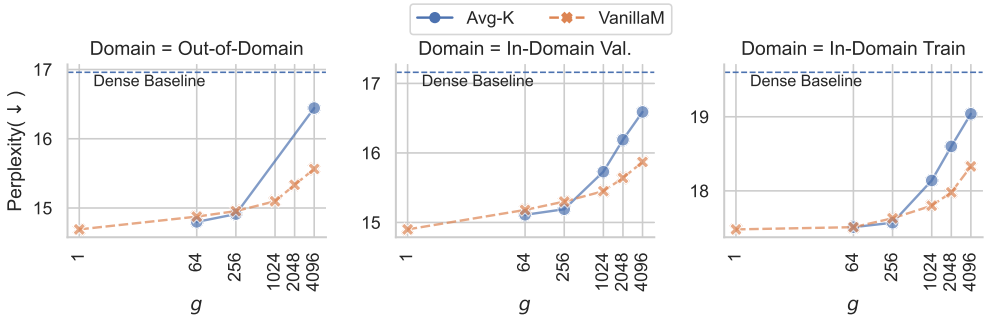



Figure 6: Perplexity performance (lower the better) of **`Avg-K`** and VanillaM across various $g$. We observe large drop in perplexity when $g$ decreases in **`Avg-K`** and less so in VanillaM; and **`Avg-K`** slightly outperform VanillaM with $g \leq 256$.

This is closest to our PKM-FFN, since their controller is essentially a gate with low-rank key table in LoRKM— $\mathbf{g}(\mathbf{x}) = (\mathbf{x} \cdot \mathbf{D}) \cdot (\mathbf{K}')^\top$, where $\mathbf{D} \in \mathbb{R}^{d \times d_\ell}$, $\mathbf{K} \in \mathbb{R}^{d_m \times d_\ell}$, and $d_\ell \ll d$. The difference is that they additionally assume the estimation from gate (and memory) could be seen as chunked into blocks and only select top-1 memory cell scored by the controller from each blocks:

$$\mathbf{y} = \sum_{i=0}^{B-1} \mathbf{g}(\mathbf{x})^{(i)}_{j^*} \cdot f(\mathbf{x} \cdot \mathbf{k}^{(i)}_{j^*}) \cdot \mathbf{v}^{(i)}_{j^*}, \text{ where } j^* = \arg\max_j \mathbf{g}(\mathbf{x})^{(i)}_j$$

Therefore, their number of active memory cells $k$ is equal to $d_m/g$.

Similar to our contrastive pair of PKM-FFN and VanillaM, we hypothesize a "vanilla" version of their methods. Memory is chunked into blocks of size $g$ — $\mathbf{K}^g = [\mathbf{K}^{(0)}; \cdots; \mathbf{K}^{(B-1)}]$ and similarly for $\mathbf{V}^g$. Then, one chooses the top-1 with $\mathbf{x} \cdot (\mathbf{K}^{(i)})^\top$. We call it **VanillaController**.

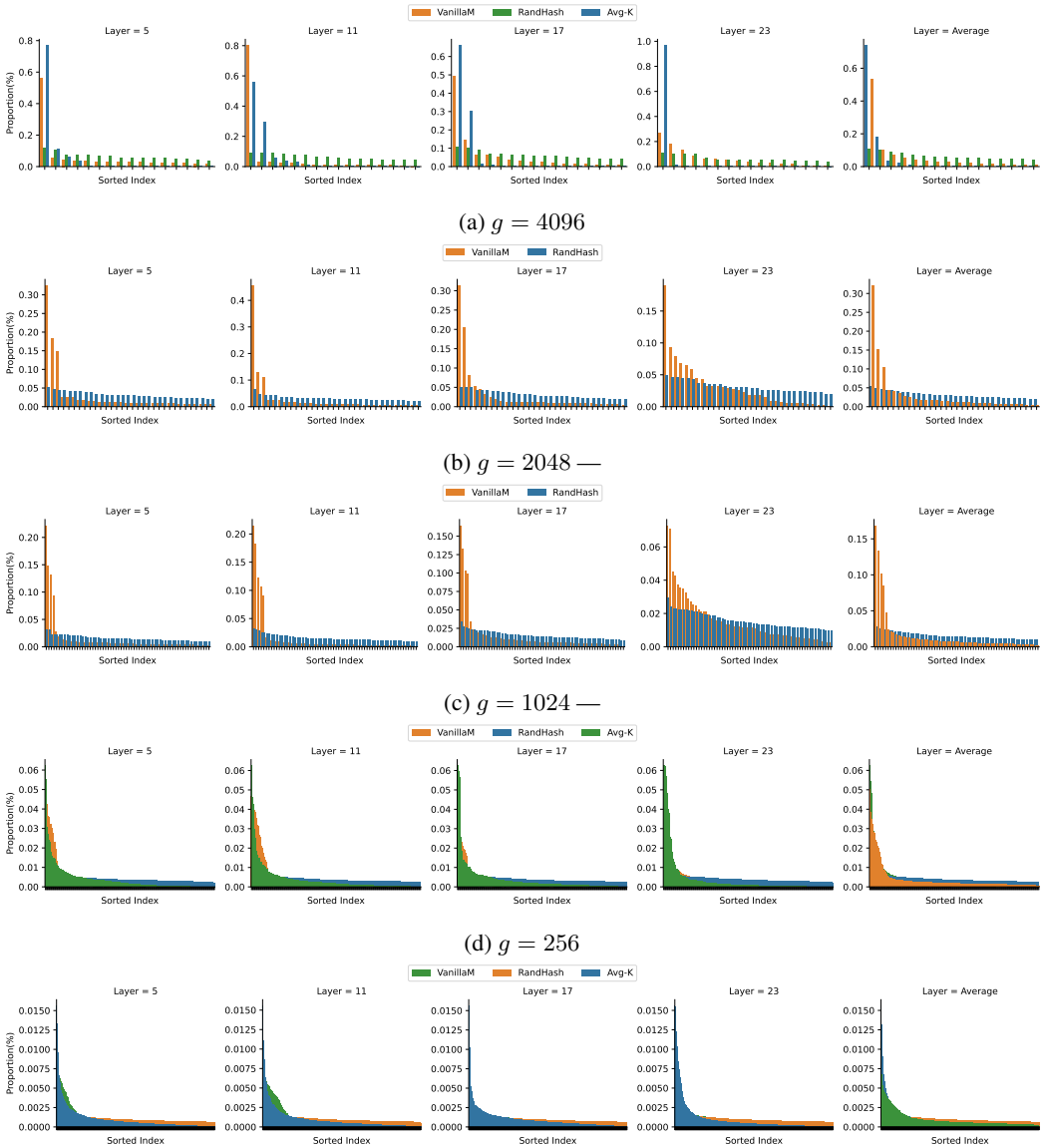
$$\mathbf{y} = \sum_{i=0}^{B-1} f(\mathbf{x} \cdot \mathbf{k}^{(i)}_{j^*}) \cdot \mathbf{v}^{(i)}_{j^*}, \text{ where } j^* = \arg\max_j \mathbf{x} \cdot (\mathbf{K}^{(i)})^\top$$

In Fig. 8, we compare VanillaController to VanillaM with $g = 1$, because the *actual* section is at the level of $g = 1$. We set $k$ in VanillaM to the one determined by equation above. We observe VanillaM outperforms VanillaController. Although the controller design as a gating function is justified (§5.2), the decision choice of "chunking memory but only select the best memory cells" seems unmotivated. Thus, we exclude this design setup from our analysis.

### D.2 ANN

Since ANN is an approximation to exact search, we propose to randomly sabotage VanillaM, which uses the exact search. Given a $k$, we randomly swap $n\%$ of the top-$k$ of memory coefficient $\mathbf{m}$ (exact search results) with non-top-$k$ values (during training and validation), and has accuracy $(100 - n)\%$ We call it **Naive-ANN**. This is meant to set up a random baseline for ANN, because different ANN techniques might make systematic mistakes, rather than a random one. However, we believe this could still serve as a proxy and shed light on how it affects performance. As we see in Fig. 9, the model quality is sensitive to the quality of ANN.

In our preliminary study, we found building data structure after every update is expensive. This leads to some critical drawback when we apply the techniques to model parameter. Although one could amortize the cost by periodically building, the outdated data structure will lead to lower accuracy. If one chooses a hyperparameter that leads to higher quality, the cost of preprocessing and the corresponding search will be even higher. What makes it worse, the current ANN methods' search either don't support speedup by using GPU, or is not very well-integrated with GPUs — slower than calculating the exact dot product with CUDA kernel.




(a) $g = 4096$

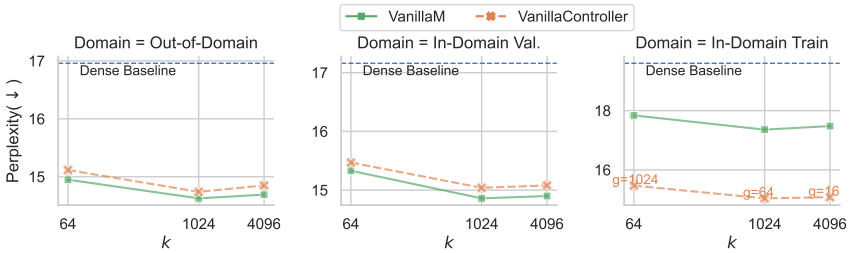
(b) $g = 2048$ —

(c) $g = 1024$ —

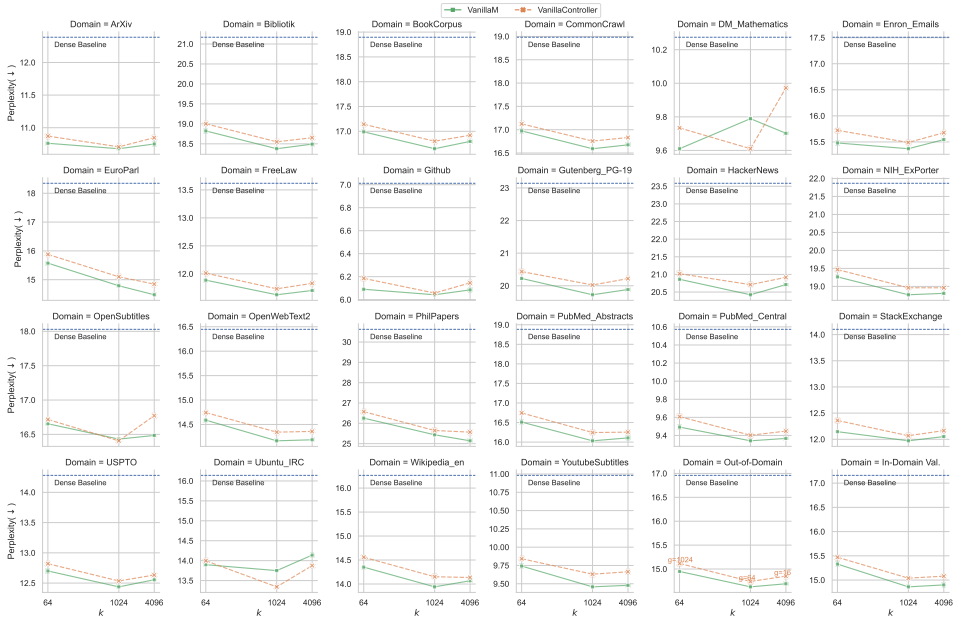
(d) $g = 256$

(e) $g = 64$

Figure 7: Load balancing of VanillaM, RandHash, **Avg-K**. The height of bar represents the proportion of memory block usage with which the memory block are sorted (in descending order).

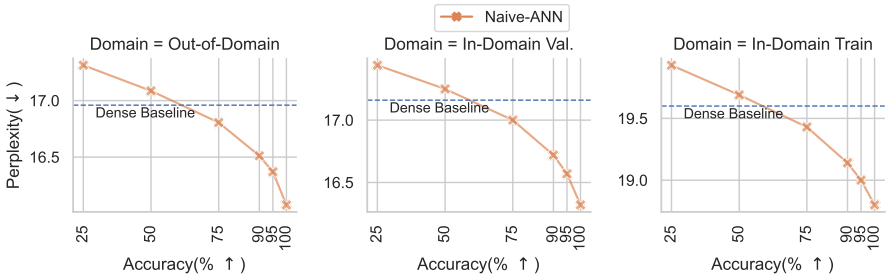



(a) Average perplexity performance (lower the better)

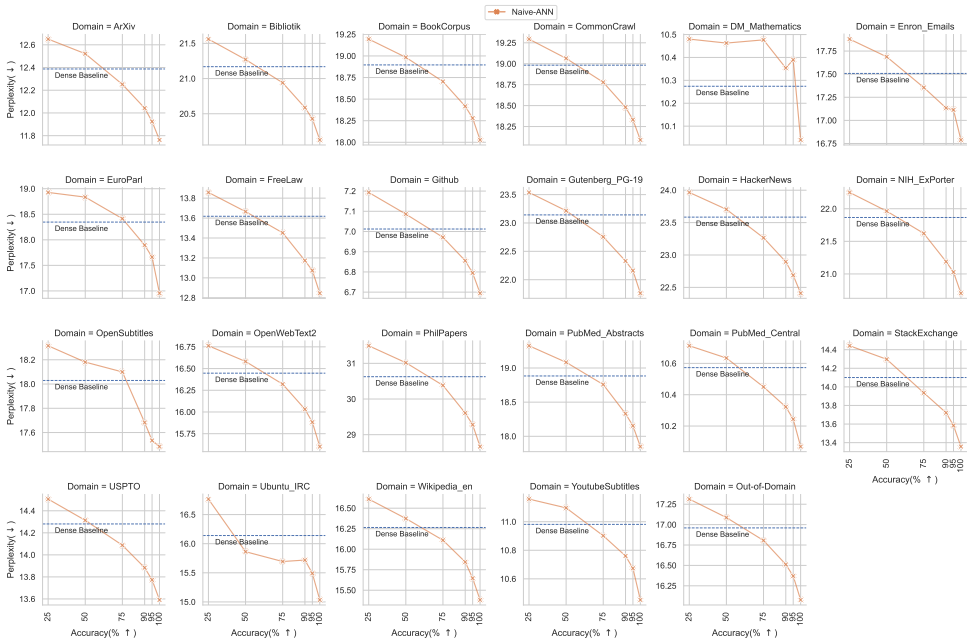



(b) Performance on individual domain in PILE perplexity (lower the better)

Figure 8: Perplexity performance (lower the better) of VanillaM (g=1) and VanillaController with $E = 16$.




(a) Average perplexity performance (lower the better)




(b) Performance on individual domain perplexity in PILE (lower the better)

Figure 9: Perplexity performance (lower the better) of Naive-ANN with $E = 4$.